```
import torch
import torch.nn as nn
import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader

from torchvision import datasets, transforms as tr

import numpy as np

import matplotlib.pyplot as plt
%matplotlib inline

# Для тёмной темы jupyter - уберите если не нужно
# plt.style.use('dark_background')

from tqdm.auto import tqdm, trange

import os
import sys
import requests
import cv2

from dataclasses import dataclass
```

```
/Users/rasanovmakar/Makar/SHAD/venv/lib/python3.12/site-packages/
tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

1. Подготовка данных (***2 балла***)

1.1 Выбор и скачивание датасета

```
dataset_folder = "/Users/rasanovmakar/Makar/SHAD/datasets"
num_images_per_split = 5

os.makedirs(dataset_folder, exist_ok=True)

# В этом цикле каждый из предложенных датасетов
#  - скачивается
#  - распаковывается
#  - отрисовывается + meta
#  - удаляется

# Предлагается посмотреть на все предложенные варианты датасетов и
затем оставить один,
```

```
#  с которым захочется работать больше всего - закомментируйте (или
удалите) все, кроме
#  выбранного, а так же закомментируйте строчки с удалением скачанного
датасета
for dataset_name in [
    # Unpaired
    # "apple2orange",
    # "summer2winter_yosemite",
    # "horse2zebra",
    "monet2photo",
    # "cezanne2photo",
    # "ukiyoe2photo",
    # "vangogh2photo",
    # Paired
    # "maps",
    # "facades",
]:
    print(f"Dataset '{dataset_name}'")
    url =
f"http://efrosgans.eecs.berkeley.edu/cyclegan/datasets/{dataset_name}.
zip"
    download_path = os.path.join(dataset_folder,
f"{dataset_name}.zip")
    target_folder = os.path.join(dataset_folder, dataset_name)

    # Чтобы bash вызовы знали соответствующие переменные
    os.environ["url"] = url
    os.environ["download_path"] = download_path
    os.environ["target_folder"] = target_folder

    print("Loading zip file...", end="")
    # Проверяем что нет такого загруженого файла
    if not os.path.isfile(download_path) or
os.path.exists(target_folder):
        # # Можно загрузить через requests библиотеку
        response = requests.get(url)
        open(download_path, "wb").write(response.content)

        # Можно загрузить через wget
        # !wget ${url} -O ${download_path}
    print(" --> done!")

    print("Unziping...", end="")
    # Распаковываем
    import shutil

    if os.path.exists(target_folder):
        shutil.rmtree(target_folder)
```

```
    !mkdir -p ${dataset_folder}
    import zipfile

    with zipfile.ZipFile(download_path, "r") as zip_ref:
        zip_ref.extractall(dataset_folder)

    print(" --> done!")

    # Удаляем zip-файл
    os.remove(download_path)


    # Meta + Отрисовка
    print(f"Provided splits: {os.listdir(target_folder)}")
    # Удобный способ быстро получить датасет картинок с лэйблами, если
картинки разложены по папкам в формате:
    # root_folder/label_name/img.jpg
    # (в нашем случае нет лэйблов, но картинки разложены в таком же
формате, просто вместо label_name идёт
    #  split_name: trainA/testA/trainB/testB)
    dataset = datasets.ImageFolder(target_folder)

    inds_to_show = {i: [] for i, _ in enumerate(dataset.classes)}
    classes_full = 0
    for dataset_ind in range(len(dataset)):
        _, split_ind = dataset[dataset_ind]
        if len(inds_to_show[split_ind]) == num_images_per_split:
            continue
        inds_to_show[split_ind].append(dataset_ind)
        if len(inds_to_show[split_ind]) == num_images_per_split:
            classes_full += 1
        if classes_full == len(dataset.classes):
            break

    for split_name in sorted(dataset.classes):
        split_ind = dataset.class_to_idx[split_name]
        print(f"Split '{split_name}' of dataset '{dataset_name}'",
end="")
        split_folder = os.path.join(target_folder, split_name)
        print(f" --> size: {len(os.listdir(split_folder))}")

        plt.subplots(1, num_images_per_split, figsize=(5 *
num_images_per_split, 5))
        plt.suptitle(f"{dataset_name} ~ {split_name}", y=0.95)
        for i, dataset_ind in enumerate(inds_to_show[split_ind]):
            plt.subplot(1, num_images_per_split, i + 1)
            plt.imshow(dataset[dataset_ind][0])
            plt.xticks([])
            plt.yticks([])
        plt.show()
```

```
    # Удаляем скачанный датасет
    # shutil.rmtree(target_folder)


    print("\n-----------------------------\n")
```

```
Dataset 'monet2photo'
Loading zip file... --> done!
Unziping... --> done!
Provided splits: ['testA', 'trainB', 'testB', 'trainA']
Split 'testA' of dataset 'monet2photo' --> size: 121
```

monet2photo – testA

```
Split 'testB' of dataset 'monet2photo' --> size: 751
```

monet2photo – testB

```
Split 'trainA' of dataset 'monet2photo' --> size: 1072
```

monet2photo – trainA

```
Split 'trainB' of dataset 'monet2photo' --> size: 6287
```

monet2photo – trainB

```
----------------------------
```

1.2 Dataset и Transforms

В папке `target_folder` находятся несколько папок со сплитами для соответствующего датасета, в каждой папке сплита находятся сами `.jpg` изображения.

Давайте составим их в удобном для нас виде в отдельные датасеты для каждого сплита без лэйблов.

Для получения картинок можно использовать

```
cv2.imread(img_path)[:, :, ::-1]  # каналы записаны в обратном порядке

# Выбранный выше и скачанный датасет
dataset_folder = "/Users/rasanovmakar/Makar/SHAD/datasets"
dataset_name = "monet2photo"
target_folder = os.path.join(dataset_folder, dataset_name)

# Класс для датасета изображений без лэйблов с применением трансформов
class ImageDatasetNoLabel(Dataset):
    def __init__(self, folder_path, transforms = None):
        super(ImageDatasetNoLabel).__init__()
        self.folder_path = folder_path
        self.transforms = transforms

        self.img_paths = sorted([
            os.path.join(folder_path, fname)
            for fname in os.listdir(folder_path)
        ])

    def __getitem__(self, index):
        img_path = self.img_paths[index]
        image = cv2.imread(img_path)[:, :, ::-1]  # BGR -> RGB

        if self.transforms is not None:
            image = self.transforms(image)

        return image
```

```
    def __len__(self):
        return len(self.img_paths)

# Удобный класс для хранения всех наших датасетов
@dataclass
class DatasetsClass:
    train_a: ImageDatasetNoLabel
    train_b: ImageDatasetNoLabel
    test_a: ImageDatasetNoLabel
    test_b: ImageDatasetNoLabel


# Все датасеты без трансформов - чтобы посчитать статистики
ds = DatasetsClass(
    train_a=ImageDatasetNoLabel(os.path.join(target_folder,
"trainA")),
    train_b=ImageDatasetNoLabel(os.path.join(target_folder,
"trainB")),
    test_a=ImageDatasetNoLabel(os.path.join(target_folder, "testA")),
    test_b=ImageDatasetNoLabel(os.path.join(target_folder, "testB")),
)

def get_channel_statistics(dataset):
    """
    Функция для получения поканальных статистик (среднее и отклонение)
по датасету
    """
    channel_sum = np.zeros(3, dtype=np.float64)
    channel_sq_sum = np.zeros(3, dtype=np.float64)
    total_pixels = 0

    for i in trange(len(dataset)):
        img = dataset[i]

        img = img.astype(np.float32) / 255.0

        pixels = img.reshape(-1, 3)

        channel_sum += pixels.sum(axis=0)
        channel_sq_sum += (pixels ** 2).sum(axis=0)
        total_pixels += pixels.shape[0]

    channel_mean = channel_sum / total_pixels
    channel_std = np.sqrt(channel_sq_sum / total_pixels - channel_mean
** 2)
    return channel_mean, channel_std

# Поканальное среднее и отклонение для A
channel_mean_a, channel_std_a = get_channel_statistics(ds.train_a)
```

```
print(channel_mean_a, channel_std_a)

# Поканальное среднее и отклонение для B
channel_mean_b, channel_std_b = get_channel_statistics(ds.train_b)
print(channel_mean_b, channel_std_b)
```

```
100%|██████████| 1072/1072 [00:01<00:00, 664.34it/s]

[0.51995585 0.51149108 0.4721689 ] [0.22548994 0.21746291 0.24333186]

100%|██████████| 6287/6287 [00:09<00:00, 636.68it/s]

[0.41227691 0.40927296 0.39264403] [0.27111535 0.24871792 0.27786204]
```

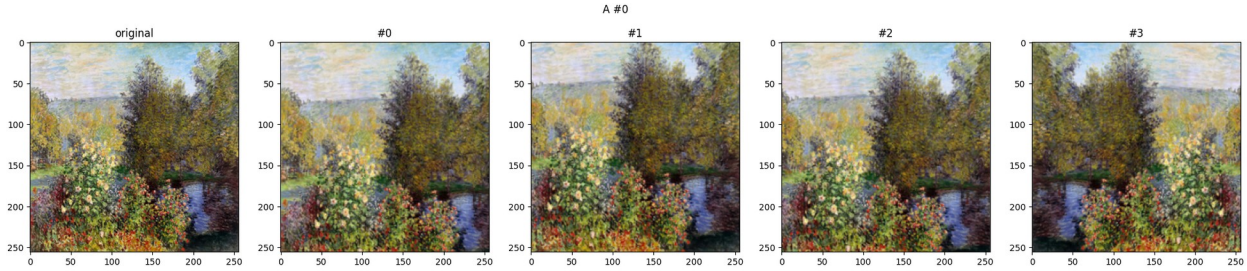
```
channel_mean_a = np.array([0.5, 0.5, 0.5], dtype=np.float32)
channel_std_a = np.array([0.5, 0.5, 0.5], dtype=np.float32)

channel_mean_b = np.array([0.5, 0.5, 0.5], dtype=np.float32)
channel_std_b = np.array([0.5, 0.5, 0.5], dtype=np.float32)

print(channel_mean_a, channel_std_a)
print(channel_mean_b, channel_std_b)
```

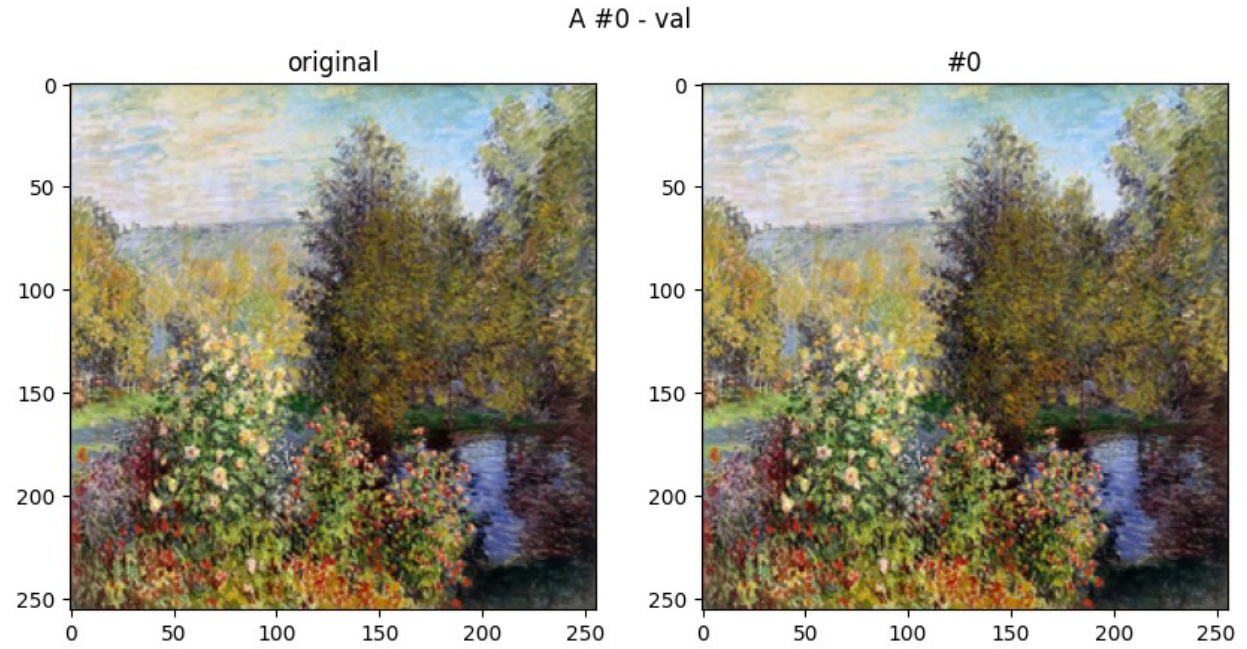
```
[0.5 0.5 0.5] [0.5 0.5 0.5]
[0.5 0.5 0.5] [0.5 0.5 0.5]
```

```
# Функция для получения train и val transform-ов, а так же функции для
де-нормализации изображения
def get_transforms(mean, std, image_size=256, resize_size=286,
p_flip=0.5):
    train_transform = tr.Compose([
        tr.ToPILImage(),
        tr.Resize((resize_size, resize_size)),
        tr.RandomCrop((image_size, image_size)),
        tr.RandomHorizontalFlip(p=p_flip),
        tr.ToTensor(),
        tr.Normalize(mean=mean, std=std),
    ])

    val_transform = tr.Compose([
        tr.ToPILImage(),
        tr.Resize((image_size, image_size)),
        tr.ToTensor(),
        tr.Normalize(mean=mean, std=std),
    ])

    def de_normalize(image, normalized=True):
        if isinstance(image, torch.Tensor):
            image = image.detach().cpu()
```

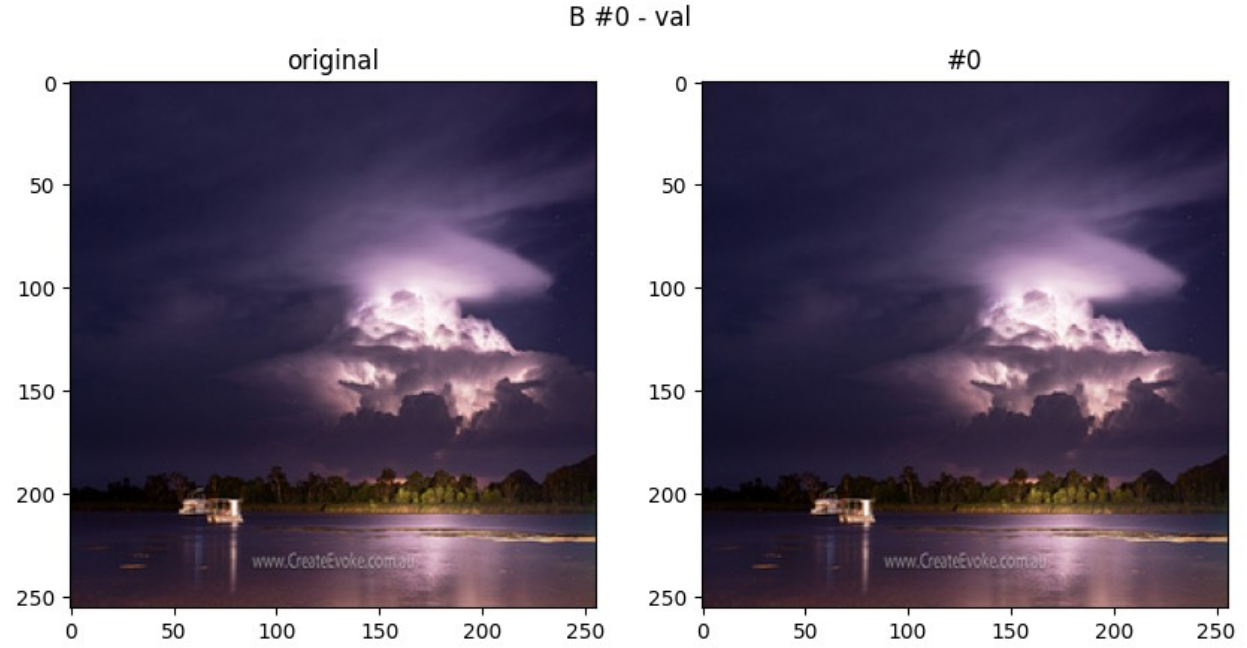
```
            if normalized:
                mean_t = torch.tensor(mean).view(3, 1, 1)
                std_t = torch.tensor(std).view(3, 1, 1)
                image = image * std_t + mean_t

            image = image.clamp(0, 1)
            image = image.permute(1, 2, 0).numpy()
        else:
            image = image.astype(np.float32)
            if image.max() > 1:
                image = image / 255.0

        return np.clip(image, 0, 1)

    return train_transform, val_transform, de_normalize


# Ваши гиперпараметры
hyperparams = dict(
    image_size=256,
    resize_size=286,
    p_flip=0.5,
)
# transform-ы для A и B
train_transform_a, val_transform_a, de_normalize_a =
get_transforms(channel_mean_a, channel_std_a, **hyperparams)
train_transform_b, val_transform_b, de_normalize_b =
get_transforms(channel_mean_b, channel_std_b, **hyperparams)


# Функция для визуализации transform-ов
def show_examples(dataset, transform, de_norm, num_per_image=3,
image_index=0, title=""):
    fig, ax = plt.subplots(1, 1 + num_per_image, figsize=(5 * (1 +
num_per_image), 5))

    image = dataset[image_index]

    plt.suptitle(title, y=0.95)

    plt.subplot(1, 1 + num_per_image, 1)
    plt.imshow(image)
    plt.title("original")

    for i in range(num_per_image):
        plt.subplot(1, 1 + num_per_image, i + 2)
        plt.title(f"#{i}")
        plt.imshow(de_norm(transform(image)))
    plt.show()
```

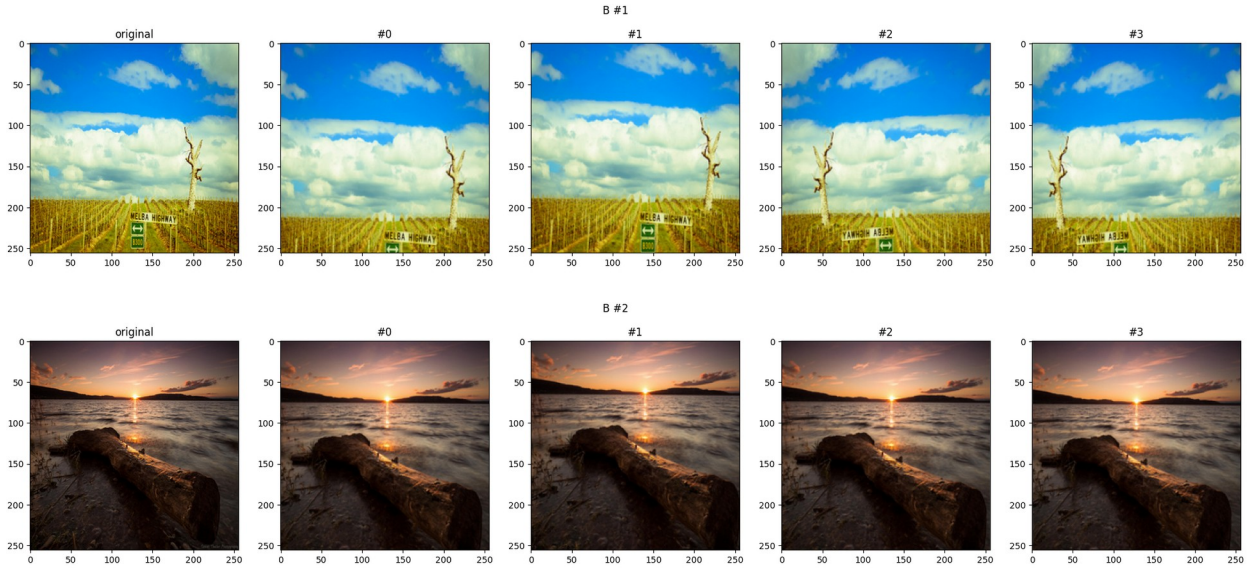
```
# Проверка на адекватность
show_examples(ds.train_a, train_transform_a, de_normalize_a,
num_per_image=4, image_index=0, title="A #0")
show_examples(ds.train_a, val_transform_a, de_normalize_a,
num_per_image=1, image_index=0, title="A #0 - val")
show_examples(ds.train_a, train_transform_a, de_normalize_a,
num_per_image=4, image_index=1, title="A #1")
show_examples(ds.train_a, train_transform_a, de_normalize_a,
num_per_image=4, image_index=2, title="A #2")
show_examples(ds.train_b, train_transform_b, de_normalize_b,
num_per_image=4, image_index=0, title="B #0")
show_examples(ds.train_b, val_transform_b, de_normalize_b,
num_per_image=1, image_index=0, title="B #0 - val")
show_examples(ds.train_b, train_transform_b, de_normalize_b,
num_per_image=4, image_index=1, title="B #1")
show_examples(ds.train_b, train_transform_b, de_normalize_b,
num_per_image=4, image_index=2, title="B #2")
```






A #1
original
#0
#1
#2
#3
A #2
original
#0
#1
#2
#3
B #0
original
#0
#1
#2
#3


B #0 - val
original
#0
www.CreateEvoke.com.au
www.CreateEvoke.com.au




```
# Все датасеты с трансформами
ds = DatasetsClass(
    train_a=ImageDatasetNoLabel(
        os.path.join(target_folder, "trainA"),
        transforms=train_transform_a,
    ),
    train_b=ImageDatasetNoLabel(
        os.path.join(target_folder, "trainB"),
        transforms=train_transform_b,
    ),
    test_a=ImageDatasetNoLabel(
        os.path.join(target_folder, "testA"),
        transforms=val_transform_a,
    ),
    test_b=ImageDatasetNoLabel(
        os.path.join(target_folder, "testB"),
        transforms=val_transform_b,
    ),
)
```

1.3 DataLoader

```
@dataclass
class DataLoadersClass:
    train_a: DataLoader
    train_b: DataLoader
    test_a: DataLoader
    test_b: DataLoader

batch_size = 4

dataloaders = DataLoadersClass(
```

```
    train_a=DataLoader(
        dataset=ds.train_a,
        batch_size=batch_size,
        shuffle=True,
        drop_last=True,
    ),
    train_b=DataLoader(
        dataset=ds.train_b,
        batch_size=batch_size,
        shuffle=True,
        drop_last=True,
    ),
    test_a=DataLoader(
        dataset=ds.test_a,
        batch_size=batch_size,
        shuffle=False,
        drop_last=True,
    ),
    test_b=DataLoader(
        dataset=ds.test_b,
        batch_size=batch_size,
        shuffle=False,
        drop_last=True,
    ),
)
```

2. Модель (***6 баллов***)

2.0 Любые вспомогательные модули и классы

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(channels, channels, kernel_size=3, stride=1,
padding=0, bias=False),
            nn.InstanceNorm2d(channels),
            nn.ReLU(inplace=True),

            nn.ReflectionPad2d(1),
            nn.Conv2d(channels, channels, kernel_size=3, stride=1,
```

```
padding=0, bias=False),
            nn.InstanceNorm2d(channels),
        )

    def forward(self, x):
        return x + self.block(x)


class ResNetGenerator(nn.Module):
    """
    Генератор в стиле статьи CycleGAN:
    """
    def __init__(self, in_channels=3, out_channels=3, ngf=64,
n_res_blocks=9):
        super().__init__()

        layers = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(in_channels, ngf, kernel_size=7, stride=1,
padding=0, bias=False),
            nn.InstanceNorm2d(ngf),
            nn.ReLU(inplace=True),
        ]

        # downsampling
        curr_channels = ngf
        for _ in range(2):
            layers += [
                nn.Conv2d(curr_channels, curr_channels * 2,
kernel_size=3, stride=2, padding=1, bias=False),
                nn.InstanceNorm2d(curr_channels * 2),
                nn.ReLU(inplace=True),
            ]
            curr_channels *= 2

        # residual blocks
        for _ in range(n_res_blocks):
            layers += [ResidualBlock(curr_channels)]

        # upsampling
        for _ in range(2):
            layers += [
                nn.ConvTranspose2d(
                    curr_channels,
                    curr_channels // 2,
                    kernel_size=3,
                    stride=2,
                    padding=1,
                    output_padding=1,
                    bias=False,
```

```
                ),
                nn.InstanceNorm2d(curr_channels // 2),
                nn.ReLU(inplace=True),
            ]
            curr_channels //= 2

        # output
        layers += [
            nn.ReflectionPad2d(3),
            nn.Conv2d(curr_channels, out_channels, kernel_size=7,
stride=1, padding=0),
            nn.Tanh(),
        ]

        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)


class PatchDiscriminator(nn.Module):
    """
    PatchGAN дискриминатор как в CycleGAN.
    """
    def __init__(self, in_channels=3, ndf=64, use_mse=True):
        super().__init__()
        out_channels = 1 if use_mse else 2

        self.model = nn.Sequential(
            nn.Conv2d(in_channels, ndf, kernel_size=4, stride=2,
padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf, ndf * 2, kernel_size=4, stride=2,
padding=1, bias=False),
            nn.InstanceNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf * 2, ndf * 4, kernel_size=4, stride=2,
padding=1, bias=False),
            nn.InstanceNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf * 4, ndf * 8, kernel_size=4, stride=1,
padding=1, bias=False),
            nn.InstanceNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf * 8, out_channels, kernel_size=4, stride=1,
padding=1),
```

```
        )

    def forward(self, x):
        return self.model(x)


def init_weights(m):
    classname = m.__class__.__name__

    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d, nn.Linear)):
        nn.init.normal_(m.weight.data, 0.0, 0.02)
        if m.bias is not None:
            nn.init.zeros_(m.bias.data)

    elif isinstance(m, (nn.InstanceNorm2d, nn.BatchNorm2d)):
        if m.weight is not None:
            nn.init.normal_(m.weight.data, 1.0, 0.02)
        if m.bias is not None:
            nn.init.zeros_(m.bias.data)
```

2.1 Архитектура сети (***3 балла***)

```
class CycleGAN(nn.Module):
    def __init__(
        self,
        in_channels=3,
        out_channels=3,
        ngf=64,
        ndf=64,
        n_res_blocks=9,
        is_mse=True,
    ):
        super(CycleGAN, self).__init__()

        self.generators = nn.ModuleDict({
            "a_to_b": ResNetGenerator(
                in_channels=in_channels,
                out_channels=out_channels,
                ngf=ngf,
                n_res_blocks=n_res_blocks,
            ),
            "b_to_a": ResNetGenerator(
                in_channels=out_channels,
                out_channels=in_channels,
                ngf=ngf,
                n_res_blocks=n_res_blocks,
            ),
        })

        self.discriminators = nn.ModuleDict({
```

```
            "a": PatchDiscriminator(in_channels=in_channels, ndf=ndf,
use_mse=is_mse),
            "b": PatchDiscriminator(in_channels=out_channels, ndf=ndf,
use_mse=is_mse),
        })

        self.apply(init_weights)

    def forward(self, imgs_a, imgs_b):
        fake_b = self.generators["a_to_b"](imgs_a)
        fake_a = self.generators["b_to_a"](imgs_b)

        rec_a = self.generators["b_to_a"](fake_b)
        rec_b = self.generators["a_to_b"](fake_a)

        a_real_pred = self.discriminators["a"](imgs_a)
        b_real_pred = self.discriminators["b"](imgs_b)
        a_fake_pred = self.discriminators["a"](fake_a)
        b_fake_pred = self.discriminators["b"](fake_b)

        return {
            "fake_a": fake_a,
            "fake_b": fake_b,
            "rec_a": rec_a,
            "rec_b": rec_b,
            "a_real_pred": a_real_pred,
            "b_real_pred": b_real_pred,
            "a_fake_pred": a_fake_pred,
            "b_fake_pred": b_fake_pred,
        }
```

2.2 Loss (***3 балла***)

$$L_{\text{cyc}}(G_{A\to B}, G_{B\to A}) = E_{a\sim A}(\| G_{B\to A}(G_{A\to B}(a)) - a \|_1) + E_{b\sim B}(\| G_{A\to B}(G_{B\to A}(b)) - b \|_1) \to \min_G$$

```
class CycleConsistencyLoss(nn.Module):
    """
    Функция ошибки, проверяющая что после двойного перехода через
генераторы изображение не изменилось
    """
    def __init__(self, reduction="mean"):
        super(CycleConsistencyLoss, self).__init__()
        self.loss = nn.L1Loss(reduction=reduction)

    def forward(self, x, x_rec):
        # Принимает на вход оригинальное изображение и изображение
после двойного перехода
        return self.loss(x_rec, x)
```

$$L_{\text{GAN}}\left(G_{A\to B}, D_B\right) = E_{b\sim B}\log D_B(b) + E_{a\sim A}\log\left(1 - D_B\left(G_{A\to B}(a)\right)\right) \to \min_G \max_D$$

```
class AdversarialLossCE(nn.Module):
    """
    Стандартная функция ошибки для minmax игры GAN-ов
    """
    def __init__(self):
        super(AdversarialLossCE, self).__init__()
        self.loss = nn.CrossEntropyLoss()

    def forward(self, real_pred, fake_pred=None):
        # Принимает на вход D_{A}(a) - real_pred и D_{A}(G(b)) -
fake_pred или наоборот
        # Может принимать только один аргумент для удобности
использования в случае
        #  обучения или генератора, или дискриминатора
        if fake_pred is None:
            target_real = torch.ones(
                real_pred.shape[0],
                real_pred.shape[2],
                real_pred.shape[3],
                dtype=torch.long,
                device=real_pred.device,
            )
            return self.loss(real_pred, target_real)

        target_real = torch.ones(
            real_pred.shape[0],
            real_pred.shape[2],
            real_pred.shape[3],
            dtype=torch.long,
            device=real_pred.device,
        )
        target_fake = torch.zeros(
            fake_pred.shape[0],
            fake_pred.shape[2],
            fake_pred.shape[3],
            dtype=torch.long,
            device=fake_pred.device,
        )

        loss_real = self.loss(real_pred, target_real)
        loss_fake = self.loss(fake_pred, target_fake)
        return 0.5 * (loss_real + loss_fake)
```

$$E_{b\sim B}¿$$

$$E_{a\sim A}¿$$

```
class AdversarialLossMSE(nn.Module):
    """
    Можно переписать не через CE, а через MSE loss на одном
предсказании, помогает со стабильностью
    """
    def __init__(self):
        super(AdversarialLossMSE, self).__init__()
        self.loss = nn.MSELoss()

    def forward(self, real_pred, fake_pred=None):
        # Принимает на вход D_{A}(a) - real_pred и D_{A}(G(b)) -
fake_pred или наоборот
        # Может принимать только один аргумент для удобности
использования в случае
        #  обучения или генератора, или дискриминатора
        if fake_pred is None:
            target_real = torch.ones_like(real_pred)
            return self.loss(real_pred, target_real)

        target_real = torch.ones_like(real_pred)
        target_fake = torch.zeros_like(fake_pred)

        loss_real = self.loss(real_pred, target_real)
        loss_fake = self.loss(fake_pred, target_fake)
        return 0.5 * (loss_real + loss_fake)
```

$$L\left(G_{A\to B}, G_{B\to A}, D_A, D_B\right) = L_{\text{GAN}}\left(G_{A\to B}, D_B\right) + L_{\text{GAN}}\left(G_{B\to A}, D_A\right) + \lambda \cdot L_{\text{cyc}}\left(G_{A\to B}, G_{B\to A}\right) \to \min_G \max_D$$

```
class FullDiscriminatorLoss(nn.Module):
    """
    Полная ошибка для дискриминатора
    """
    def __init__(self, is_mse=True):
        super(FullDiscriminatorLoss, self).__init__()
        self.adversarial_loss_func = AdversarialLossMSE() if is_mse
else AdversarialLossCE()

    def forward(
        self,
        a_real_pred,
        a_fake_pred,
        b_real_pred,
        b_fake_pred,
    ):
        loss_a = self.adversarial_loss_func(a_real_pred, a_fake_pred)
        loss_b = self.adversarial_loss_func(b_real_pred, b_fake_pred)
        return loss_a + loss_b
```

```
class FullGeneratorLoss(nn.Module):
    """
    Полная ошибка для генератора
    """
    def __init__(self, lambda_value=10., is_mse=True,
use_identity_loss=False, identity_lambda=0.5):
        super(FullGeneratorLoss, self).__init__()
        self.adversarial_loss_func = AdversarialLossMSE() if is_mse
else AdversarialLossCE()
        self.cycle_consistency_loss_func = CycleConsistencyLoss()
        self.identity_loss_func = nn.L1Loss()
        self.lambda_value = lambda_value
        self.use_identity_loss = use_identity_loss
        self.identity_lambda = identity_lambda

    def forward(
        self,
        a_fake_pred,
        b_fake_pred,
        imgs_a,
        imgs_b,
        rec_a,
        rec_b,
        idt_a=None,
        idt_b=None,
    ):
        loss_gan_a_to_b = self.adversarial_loss_func(b_fake_pred)
        loss_gan_b_to_a = self.adversarial_loss_func(a_fake_pred)

        loss_cycle_a = self.cycle_consistency_loss_func(imgs_a, rec_a)
        loss_cycle_b = self.cycle_consistency_loss_func(imgs_b, rec_b)
        loss_cycle = loss_cycle_a + loss_cycle_b

        loss = loss_gan_a_to_b + loss_gan_b_to_a + self.lambda_value *
loss_cycle

        if self.use_identity_loss and idt_a is not None and idt_b is
not None:
            loss_idt_a = self.identity_loss_func(idt_a, imgs_a)
            loss_idt_b = self.identity_loss_func(idt_b, imgs_b)
            loss += self.lambda_value * self.identity_lambda *
(loss_idt_a + loss_idt_b)

        return loss
```

3. Подготовка обучения (**2 балла**)

3.1 Шаг обучения дискриминатора

```
def train_discriminators(model, opt_d, loader_a, loader_b,
criterion_d):
    model.train()
    losses_tr = []

    iter_a = iter(loader_a)
    iter_b = iter(loader_b)
    batches_per_epoch = min(len(iter_a), len(iter_b))

    for _ in trange(batches_per_epoch):
        imgs_a = next(iter_a).to(device)
        imgs_b = next(iter_b).to(device)

        opt_d.zero_grad()

        with torch.no_grad():
            fake_b = model.generators["a_to_b"](imgs_a)
            fake_a = model.generators["b_to_a"](imgs_b)

        a_real_pred = model.discriminators["a"](imgs_a)
        b_real_pred = model.discriminators["b"](imgs_b)
        a_fake_pred = model.discriminators["a"](fake_a.detach())
        b_fake_pred = model.discriminators["b"](fake_b.detach())

        loss = criterion_d(
            a_real_pred=a_real_pred,
            a_fake_pred=a_fake_pred,
            b_real_pred=b_real_pred,
            b_fake_pred=b_fake_pred,
        )

        loss.backward()
        opt_d.step()
        losses_tr.append(loss.item())

    return model, opt_d, np.mean(losses_tr)
```

3.2 Шаг обучения генератора

```
def train_generators(model, opt_g, loader_a, loader_b, criterion_g):
    model.train()
    losses_tr = []

    iter_a = iter(loader_a)
```

```
    iter_b = iter(loader_b)
    batches_per_epoch = min(len(iter_a), len(iter_b))

    for _ in trange(batches_per_epoch):
        imgs_a = next(iter_a).to(device)
        imgs_b = next(iter_b).to(device)

        opt_g.zero_grad()

        fake_b = model.generators["a_to_b"](imgs_a)
        fake_a = model.generators["b_to_a"](imgs_b)

        rec_a = model.generators["b_to_a"](fake_b)
        rec_b = model.generators["a_to_b"](fake_a)

        a_fake_pred = model.discriminators["a"](fake_a)
        b_fake_pred = model.discriminators["b"](fake_b)

        # optional identity
        idt_a = model.generators["b_to_a"](imgs_a)
        idt_b = model.generators["a_to_b"](imgs_b)

        loss = criterion_g(
            a_fake_pred=a_fake_pred,
            b_fake_pred=b_fake_pred,
            imgs_a=imgs_a,
            imgs_b=imgs_b,
            rec_a=rec_a,
            rec_b=rec_b,
            idt_a=idt_a,
            idt_b=idt_b,
        )

        loss.backward()
        opt_g.step()
        losses_tr.append(loss.item())

    return model, opt_g, np.mean(losses_tr)
```

3.3 Шаг валидации

```
from collections import defaultdict

def val(model, loader_a, loader_b, criterion_d, criterion_g):
    model.eval()

    val_data = defaultdict(list)

    with torch.no_grad():
        iter_a = iter(loader_a)
```

```
        iter_b = iter(loader_b)
        batches_per_epoch = min(len(iter_a), len(iter_b))

        for _ in trange(batches_per_epoch):
            imgs_a = next(iter_a).to(device)
            imgs_b = next(iter_b).to(device)

            fake_b = model.generators["a_to_b"](imgs_a)
            fake_a = model.generators["b_to_a"](imgs_b)

            rec_a = model.generators["b_to_a"](fake_b)
            rec_b = model.generators["a_to_b"](fake_a)

            a_real_pred = model.discriminators["a"](imgs_a)
            b_real_pred = model.discriminators["b"](imgs_b)
            a_fake_pred = model.discriminators["a"](fake_a)
            b_fake_pred = model.discriminators["b"](fake_b)

            idt_a = model.generators["b_to_a"](imgs_a)
            idt_b = model.generators["a_to_b"](imgs_b)

            loss_d = criterion_d(
                a_real_pred=a_real_pred,
                a_fake_pred=a_fake_pred,
                b_real_pred=b_real_pred,
                b_fake_pred=b_fake_pred,
            )

            loss_g = criterion_g(
                a_fake_pred=a_fake_pred,
                b_fake_pred=b_fake_pred,
                imgs_a=imgs_a,
                imgs_b=imgs_b,
                rec_a=rec_a,
                rec_b=rec_b,
                idt_a=idt_a,
                idt_b=idt_b,
            )

            val_data["loss D"].append(loss_d.item())
            val_data["loss G"].append(loss_g.item())

            # Оставлю для вас мой кусочек логирования для
визуализации, думаю по аналогии
            #  разберётесь что предполагалось в каких переменных
            is_mse_pred = a_real_pred.shape[1] == 1

            if is_mse_pred:
                a_real_pred = a_real_pred.mean(dim=(1, 2, 3))
                b_real_pred = b_real_pred.mean(dim=(1, 2, 3))
```

```
                a_fake_pred = a_fake_pred.mean(dim=(1, 2, 3))
                b_fake_pred = b_fake_pred.mean(dim=(1, 2, 3))
            else:
                a_real_pred = F.softmax(a_real_pred, dim=1)[:,
1].mean(dim=(1, 2))
                b_real_pred = F.softmax(b_real_pred, dim=1)[:,
1].mean(dim=(1, 2))
                a_fake_pred = F.softmax(a_fake_pred, dim=1)[:,
1].mean(dim=(1, 2))
                b_fake_pred = F.softmax(b_fake_pred, dim=1)[:,
1].mean(dim=(1, 2))


            val_data["real pred
A"].extend(a_real_pred.cpu().detach().tolist())
            val_data["real pred
B"].extend(b_real_pred.cpu().detach().tolist())
            val_data["fake pred
A"].extend(a_fake_pred.cpu().detach().tolist())
            val_data["fake pred
B"].extend(b_fake_pred.cpu().detach().tolist())

        val_data["loss D"] = np.mean(val_data["loss D"])
        val_data["loss G"] = np.mean(val_data["loss G"])

    return val_data
```

3.4 Визуализация сгенерированного

```
def draw_imgs(model, num_images, loader_a, loader_b, de_norm_a,
de_norm_b):
    model.eval()
    with torch.no_grad():
        imgs_a = next(iter(loader_a))[:num_images].to(device)
        imgs_b = next(iter(loader_b))[:num_images].to(device)

        fake_a = model.generators["b_to_a"](imgs_b)
        fake_b = model.generators["a_to_b"](imgs_a)
        rec_a = model.generators["b_to_a"](fake_b)
        rec_b = model.generators["a_to_b"](fake_a)

        # Draw num_images examples for A
        fig, ax = plt.subplots(num_images, 3, figsize=(25, 15))
        plt.suptitle("Images from A", y=0.92)

        for ind in range(num_images):
            plt.subplot(num_images, 3, ind * 3 + 1)
            plt.title("Original from A")
            plt.imshow(de_norm_a(imgs_a[ind], normalized=True))
```

```
            plt.xticks([])
            plt.yticks([])

            plt.subplot(num_images, 3, ind * 3 + 2)
            plt.title("Translated to B")
            plt.imshow(de_norm_b(fake_b[ind], normalized=True))
            plt.xticks([])
            plt.yticks([])

            plt.subplot(num_images, 3, ind * 3 + 3)
            plt.title("Reconstructed A")
            plt.imshow(de_norm_a(rec_a[ind], normalized=True))
            plt.xticks([])
            plt.yticks([])

        # Draw num_images examples for B
        fig, ax = plt.subplots(num_images, 3, figsize=(25, 15))
        plt.suptitle("Images from B", y=0.92)

        for ind in range(num_images):
            plt.subplot(num_images, 3, ind * 3 + 1)
            plt.title("Original from B")
            plt.imshow(de_norm_b(imgs_b[ind], normalized=True))
            plt.xticks([])
            plt.yticks([])

            plt.subplot(num_images, 3, ind * 3 + 2)
            plt.title("Translated to A")
            plt.imshow(de_norm_a(fake_a[ind], normalized=True))
            plt.xticks([])
            plt.yticks([])

            plt.subplot(num_images, 3, ind * 3 + 3)
            plt.title("Reconstructed B")
            plt.imshow(de_norm_b(rec_b[ind], normalized=True))
            plt.xticks([])
            plt.yticks([])

        plt.show()
```

3.5 Цикл обучения

```
from IPython.display import clear_output
import warnings

def get_model_name(chkp_folder, model_name=None):
    # Выбираем имя чекпоинта для сохранения
    if model_name is None:
        if os.path.exists(chkp_folder):
```

```
            num_starts = len(os.listdir(chkp_folder)) + 1
        else:
            num_starts = 1
        model_name = f'model#{num_starts}'
    else:
        if "#" not in model_name:
            model_name += "#0"
    changed = False
    while os.path.exists(os.path.join(chkp_folder, model_name +
'.pt')):
        model_name, ind = model_name.split("#")
        model_name += f"#{int(ind) + 1}"
        changed=True
    if changed:
        warnings.warn(f"Selected model_name was used already! To avoid
possible overwrite - model_name changed to {model_name}")
    return model_name


def get_lr(optimizer):
    for param_group in optimizer.param_groups:
        return param_group['lr']


def learning_loop(
    model,
    optimizer_g,
    g_iters_per_epoch,
    optimizer_d,
    d_iters_per_epoch,
    train_loader_a,
    train_loader_b,
    val_loader_a,
    val_loader_b,
    criterion_d,
    criterion_g,
    de_norm_a,
    de_norm_b,
    scheduler_d=None,
    scheduler_g=None,
    min_lr=None,
    epochs=10,
    val_every=1,
    draw_every=1,
    model_name=None,
    chkp_folder="./chkps",
    images_per_validation=3,
    plots=None,
    starting_epoch=0,
):
```

```
    model_name = get_model_name(chkp_folder, model_name)

    if plots is None:
        plots = {
            'train G': [],
            'train D': [],
            'val D': [],
            'val G': [],
            "lr G": [],
            "lr D": [],
            "hist real A": [],
            "hist gen A": [],
            "hist real B": [],
            "hist gen B": [],
        }

    for epoch in np.arange(1, epochs+1) + starting_epoch:
        print(f'#{epoch}/{epochs}:')

        plots['lr G'].append(get_lr(optimizer_g))
        plots['lr D'].append(get_lr(optimizer_d))

        # train discriminators
        print(f"train discriminators ({d_iters_per_epoch} times)")
        loss_d = []
        for _ in range(d_iters_per_epoch):
            model, optimizer_d, loss = train_discriminators(model,
optimizer_d, train_loader_a, train_loader_b, criterion_d)
            loss_d.append(loss)
        plots['train D'].extend(loss_d)

        # train generators
        print(f"train generators ({g_iters_per_epoch} times)")
        loss_g = []
        for _ in range(g_iters_per_epoch):
            model, optimizer_g, loss = train_generators(model,
optimizer_g, train_loader_a, train_loader_b, criterion_g)
            loss_g.append(loss)
        plots['train G'].extend(loss_g)

        if not (epoch % val_every):
            print("validate")
            val_data = val(model, val_loader_a, val_loader_b,
criterion_d, criterion_g)
            plots['val D'].append(val_data["loss D"])
            plots['val G'].append(val_data["loss G"])
            plots['hist real A'].append(val_data["real pred A"])
            plots['hist gen A'].append(val_data["fake pred A"])
            plots['hist real B'].append(val_data["real pred B"])
            plots['hist gen B'].append(val_data["fake pred B"])
```

```
            # Сохраняем модель
            if not os.path.exists(chkp_folder):
                os.makedirs(chkp_folder)
            torch.save(
                {
                    'epoch': epoch,
                    'model_state_dict': model.state_dict(),
                    'optimizer_d_state_dict':
optimizer_d.state_dict(),
                    'optimizer_g_state_dict':
optimizer_g.state_dict(),
                    'scheduler_d_state_dict':
scheduler_d.state_dict(),
                    'scheduler_g_state_dict':
scheduler_g.state_dict(),
                    'plots': plots,
                },
                os.path.join(chkp_folder, model_name + '.pt'),
            )

            # Шедулинг
            if scheduler_d:
                try:
                    scheduler_d.step()
                except:
                    scheduler_d.step(loss_d)
            if scheduler_g:
                try:
                    scheduler_g.step()
                except:
                    scheduler_g.step(loss_g)

        if not (epoch % draw_every):
            clear_output(True)

            hh = 2
            ww = 2
            plt_ind = 1
            fig, ax = plt.subplots(hh, ww, figsize=(25, 12))
            fig.suptitle(f'#{epoch}/{epochs}:')

            plt.subplot(hh, ww, plt_ind)
            plt.title('discriminators losses')
            d_plot_step = 1. / d_iters_per_epoch
            plt.plot(np.arange(d_plot_step, epoch + d_plot_step,
d_plot_step), plots['train D'], 'r.-', label='train', alpha=0.7)
            plt.plot(np.arange(1, epoch + 1), plots['val D'], 'g.-',
label='val', alpha=0.7)
```

```
            plt.grid()
            plt.legend()
            plt_ind += 1

            plt.subplot(hh, ww, plt_ind)
            plt.title('generators losses')
            g_plot_step = 1. / g_iters_per_epoch
            plt.plot(np.arange(g_plot_step, epoch + g_plot_step,
g_plot_step), plots['train G'], 'r.-', label='train', alpha=0.7)
            plt.plot(np.arange(1, epoch + 1), plots['val G'], 'g.-',
label='val', alpha=0.7)
            plt.grid()
            plt.legend()
            plt_ind += 1

            # plt.subplot(hh, ww, plt_ind)
            # plt.title('learning rates')
            # plt.plot(plots["lr D"], 'b.-', label='lr discriminator',
alpha=0.7)
            # plt.plot(plots["lr G"], 'm.-', label='lr generator',
alpha=0.7)
            # plt.legend()
            # plt_ind += 1

            plt.subplot(hh, ww, plt_ind)
            plt.title("Discriminator A predictions")
            plt.hist(plots["hist real A"][-1], bins=50, density=True,
label="real", color="green", alpha=0.7)
            plt.hist(plots["hist gen A"][-1], bins=50, density=True,
label="generated", color="red", alpha=0.7)
            plt.xlim((-0.05, 1.05))
            plt.xticks(ticks=np.arange(0, 1.05, 0.1))
            plt.legend()
            plt_ind += 1

            plt.subplot(hh, ww, plt_ind)
            plt.title("Discriminator B predictions")
            plt.hist(plots["hist real B"][-1], bins=50, density=True,
label="real", color="green", alpha=0.7)
            plt.hist(plots["hist gen B"][-1], bins=50, density=True,
label="generated", color="red", alpha=0.7)
            plt.xlim((-0.05, 1.05))
            plt.xticks(ticks=np.arange(0, 1.05, 0.1))
            plt.legend()
            plt_ind += 1

            plt.show()

            draw_imgs(model, images_per_validation, val_loader_a,
```

```
val_loader_b, de_norm_a, de_norm_b)


        if min_lr and get_lr(optimizer_d) <= min_lr:
            print(f'Learning process ended with early stop for
discriminator after epoch {epoch}')
            break

        if min_lr and get_lr(optimizer_g) <= min_lr:
            print(f'Learning process ended with early stop for
generator after epoch {epoch}')
            break

    return model, optimizer_d, optimizer_g, plots
```

4. Обучение (***2 балла***)

4.1 Инициализация модели и оптимайзера

```
from collections import defaultdict
from termcolor import colored


def beautiful_int(i):
    i = str(i)
    return ".".join(reversed([i[max(j, 0):j+3] for j in range(len(i) -
3, -3, -3)]))


# Подсчёт числа параметров в нашей модели
def model_num_params(model, verbose_all=True,
verbose_only_learnable=False):
    sum_params = 0
    sum_learnable_params = 0
    submodules = defaultdict(lambda : [0, 0])
    for name, param in model.named_parameters():
        num_params = param.numel()
        if verbose_all or (verbose_only_learnable and
param.requires_grad):
            print(
                colored(
                    '{: <65} ~  {: <9} params ~ grad: {}'.format(
                        name,
                        beautiful_int(num_params),
                        param.requires_grad,
                    ),
                    {True: "green", False: "red"}
```

```
[param.requires_grad],
                )
            )
            sum_params += num_params
            sm = name.split(".")[0]
            submodules[sm][0] += num_params
            if param.requires_grad:
                sum_learnable_params += num_params
                submodules[sm][1] += num_params
    print(
        f'\nIn total:\n  - {beautiful_int(sum_params)} params\n  -
{beautiful_int(sum_learnable_params)} learnable params'
    )

    for sm, v in submodules.items():
        print(
            f"\n . {sm}:\n .   - {beautiful_int(submodules[sm][0])}
params\n .   - {beautiful_int(submodules[sm][1])} learnable params"
        )
    return sum_params, sum_learnable_params

device = torch.device("mps") if torch.backends.mps.is_available() else
torch.device("cpu")
print(device)

def create_model_and_optimizer(model_class, model_params, lr=1e-3,
betas=(0.5, 0.999), weight_decay=0.0, device=device):
    model = model_class(**model_params)
    model = model.to(device)

    optimizer_d = torch.optim.Adam(
        model.discriminators.parameters(),
        lr=lr,
        betas=betas,
        weight_decay=weight_decay,
    )
    optimizer_g = torch.optim.Adam(
        model.generators.parameters(),
        lr=lr,
        betas=betas,
        weight_decay=weight_decay,
    )
    return model, optimizer_d, optimizer_g
```

```
mps
```

4.2 Фактическое обучение

```
results = []
```

```
model, optimizer_d, optimizer_g = create_model_and_optimizer(
    model_class=CycleGAN,
    model_params=dict(
        in_channels=3,
        out_channels=3,
        ngf=64,
        ndf=64,
        n_res_blocks=9,
        is_mse=True,
    ),
    lr=2e-4,
    betas=(0.5, 0.999),
    device=device,
)


scheduler_d = torch.optim.lr_scheduler.LambdaLR(
    optimizer_d,
    lr_lambda=lambda epoch: 1.0
)
scheduler_g = torch.optim.lr_scheduler.LambdaLR(
    optimizer_g,
    lr_lambda=lambda epoch: 1.0
)

criterion_d = FullDiscriminatorLoss(is_mse=True)
criterion_g = FullGeneratorLoss(
    lambda_value=10.,
    is_mse=True,
    use_identity_loss=False,
    identity_lambda=0.5,
)

sum_params, sum_learnable_params = model_num_params(model)
```

```
generators.a_to_b.model.1.weight                                ~
9.408     params ~ grad: True
generators.a_to_b.model.4.weight                                ~
73.728    params ~ grad: True
generators.a_to_b.model.7.weight                                ~
294.912   params ~ grad: True
generators.a_to_b.model.10.block.1.weight                       ~
589.824   params ~ grad: True
generators.a_to_b.model.10.block.5.weight                       ~
589.824   params ~ grad: True
generators.a_to_b.model.11.block.1.weight                       ~
589.824   params ~ grad: True
generators.a_to_b.model.11.block.5.weight                       ~
589.824   params ~ grad: True
generators.a_to_b.model.12.block.1.weight                       ~
```

```
589.824   params ~ grad: True
generators.a_to_b.model.12.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.13.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.13.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.14.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.14.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.15.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.15.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.16.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.16.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.17.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.17.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.18.block.1.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.18.block.5.weight                          ~
589.824   params ~ grad: True
generators.a_to_b.model.19.weight                                  ~
294.912   params ~ grad: True
generators.a_to_b.model.22.weight                                  ~
73.728    params ~ grad: True
generators.a_to_b.model.26.weight                                  ~
9.408     params ~ grad: True
generators.a_to_b.model.26.bias                                    ~  3
params ~ grad: True
generators.b_to_a.model.1.weight                                   ~
9.408     params ~ grad: True
generators.b_to_a.model.4.weight                                   ~
73.728    params ~ grad: True
generators.b_to_a.model.7.weight                                   ~
294.912   params ~ grad: True
generators.b_to_a.model.10.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.10.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.11.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.11.block.5.weight                          ~
589.824   params ~ grad: True
```

```
generators.b_to_a.model.12.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.12.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.13.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.13.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.14.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.14.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.15.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.15.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.16.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.16.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.17.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.17.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.18.block.1.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.18.block.5.weight                          ~
589.824   params ~ grad: True
generators.b_to_a.model.19.weight                                  ~
294.912   params ~ grad: True
generators.b_to_a.model.22.weight                                  ~
73.728    params ~ grad: True
generators.b_to_a.model.26.weight                                  ~
9.408     params ~ grad: True
generators.b_to_a.model.26.bias                                    ~  3
params ~ grad: True
discriminators.a.model.0.weight                                    ~
3.072     params ~ grad: True
discriminators.a.model.0.bias                                      ~
64        params ~ grad: True
discriminators.a.model.2.weight                                    ~
131.072   params ~ grad: True
discriminators.a.model.5.weight                                    ~
524.288   params ~ grad: True
discriminators.a.model.8.weight                                    ~
2.097.152 params ~ grad: True
discriminators.a.model.11.weight                                   ~
8.192     params ~ grad: True
discriminators.a.model.11.bias                                     ~  1
```

```
params ~ grad: True
discriminators.b.model.0.weight                                    ~
3.072     params ~ grad: True
discriminators.b.model.0.bias                                      ~
64        params ~ grad: True
discriminators.b.model.2.weight                                    ~
131.072   params ~ grad: True
discriminators.b.model.5.weight                                    ~
524.288   params ~ grad: True
discriminators.b.model.8.weight                                    ~
2.097.152 params ~ grad: True
discriminators.b.model.11.weight                                   ~
8.192     params ~ grad: True
discriminators.b.model.11.bias                                     ~  1
params ~ grad: True

In total:
  - 28.273.544 params
  - 28.273.544 learnable params

 . generators:
 .   - 22.745.862 params
 .   - 22.745.862 learnable params

 . discriminators:
 .   - 5.527.682 params
 .   - 5.527.682 learnable params



model

CycleGAN(
  (generators): ModuleDict(
    (a_to_b): ResNetGenerator(
      (model): Sequential(
        (0): ReflectionPad2d((3, 3, 3, 3))
        (1): Conv2d(3, 64, kernel_size=(7, 7), stride=(1, 1),
bias=False)
        (2): InstanceNorm2d(64, eps=1e-05, momentum=0.1, affine=False,
track_running_stats=False)
        (3): ReLU(inplace=True)
        (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (5): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (6): ReLU(inplace=True)
        (7): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (8): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
```

```
affine=False, track_running_stats=False)
        (9): ReLU(inplace=True)
        (10): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (11): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (12): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (13): ResidualBlock(
          (block): Sequential(
```

```
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (14): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (15): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (16): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
```

```
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (17): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (18): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (19): ConvTranspose2d(256, 128, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), output_padding=(1, 1), bias=False)
        (20): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (21): ReLU(inplace=True)
        (22): ConvTranspose2d(128, 64, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), output_padding=(1, 1), bias=False)
        (23): InstanceNorm2d(64, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (24): ReLU(inplace=True)
```

```
        (25): ReflectionPad2d((3, 3, 3, 3))
        (26): Conv2d(64, 3, kernel_size=(7, 7), stride=(1, 1))
        (27): Tanh()
      )
    )
    (b_to_a): ResNetGenerator(
      (model): Sequential(
        (0): ReflectionPad2d((3, 3, 3, 3))
        (1): Conv2d(3, 64, kernel_size=(7, 7), stride=(1, 1),
bias=False)
        (2): InstanceNorm2d(64, eps=1e-05, momentum=0.1, affine=False,
track_running_stats=False)
        (3): ReLU(inplace=True)
        (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (5): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (6): ReLU(inplace=True)
        (7): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (8): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (9): ReLU(inplace=True)
        (10): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (11): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
```

```
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (12): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (13): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (14): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
```

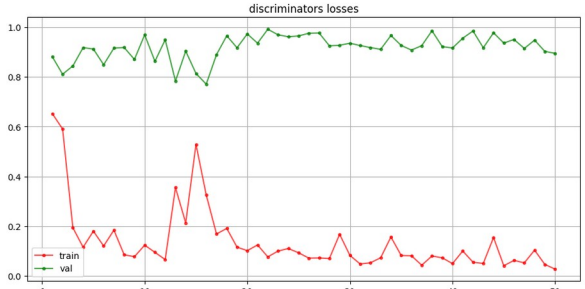
```
        (15): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (16): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (17): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (18): ResidualBlock(
          (block): Sequential(
            (0): ReflectionPad2d((1, 1, 1, 1))
            (1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
```

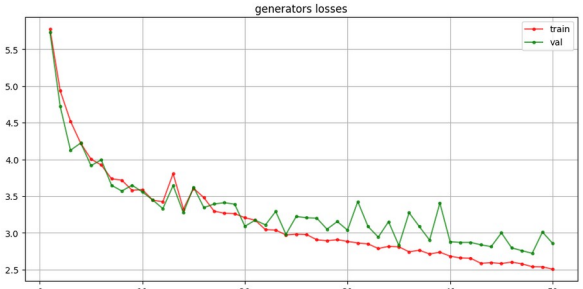
```
bias=False)
            (2): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
            (3): ReLU(inplace=True)
            (4): ReflectionPad2d((1, 1, 1, 1))
            (5): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
bias=False)
            (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
          )
        )
        (19): ConvTranspose2d(256, 128, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), output_padding=(1, 1), bias=False)
        (20): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (21): ReLU(inplace=True)
        (22): ConvTranspose2d(128, 64, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), output_padding=(1, 1), bias=False)
        (23): InstanceNorm2d(64, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (24): ReLU(inplace=True)
        (25): ReflectionPad2d((3, 3, 3, 3))
        (26): Conv2d(64, 3, kernel_size=(7, 7), stride=(1, 1))
        (27): Tanh()
      )
    )
  )
  (discriminators): ModuleDict(
    (a): PatchDiscriminator(
      (model): Sequential(
        (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1))
        (1): LeakyReLU(negative_slope=0.2, inplace=True)
        (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1), bias=False)
        (3): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (4): LeakyReLU(negative_slope=0.2, inplace=True)
        (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1), bias=False)
        (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (7): LeakyReLU(negative_slope=0.2, inplace=True)
        (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(1, 1),
padding=(1, 1), bias=False)
        (9): InstanceNorm2d(512, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (10): LeakyReLU(negative_slope=0.2, inplace=True)
        (11): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1),
```

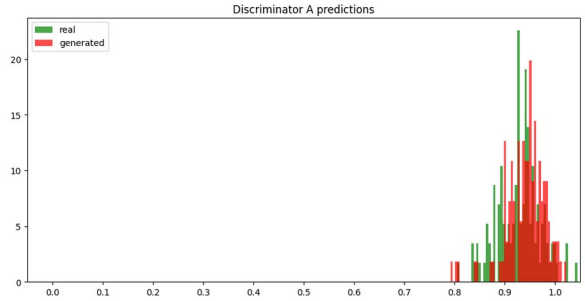
```
padding=(1, 1))
      )
    )
    (b): PatchDiscriminator(
      (model): Sequential(
        (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1))
        (1): LeakyReLU(negative_slope=0.2, inplace=True)
        (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1), bias=False)
        (3): InstanceNorm2d(128, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (4): LeakyReLU(negative_slope=0.2, inplace=True)
        (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2),
padding=(1, 1), bias=False)
        (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (7): LeakyReLU(negative_slope=0.2, inplace=True)
        (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(1, 1),
padding=(1, 1), bias=False)
        (9): InstanceNorm2d(512, eps=1e-05, momentum=0.1,
affine=False, track_running_stats=False)
        (10): LeakyReLU(negative_slope=0.2, inplace=True)
        (11): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1),
padding=(1, 1))
      )
    )
  )
)

%%time
model, optimizer_d, optimizer_g, plots = learning_loop(
    model = model,
    optimizer_g = optimizer_g,
    g_iters_per_epoch = 1,
    optimizer_d = optimizer_d,
    d_iters_per_epoch = 1,
    train_loader_a = dataloaders.train_a,
    train_loader_b = dataloaders.train_b,
    val_loader_a = dataloaders.test_a,
    val_loader_b = dataloaders.test_b,
    criterion_d = criterion_d,
    criterion_g = criterion_g,
    scheduler_g = scheduler_g,
    scheduler_d = scheduler_d,
    de_norm_a = de_normalize_a,
    de_norm_b = de_normalize_b,
    epochs = 50,
    min_lr = 1e-6,
    val_every = 1,
```

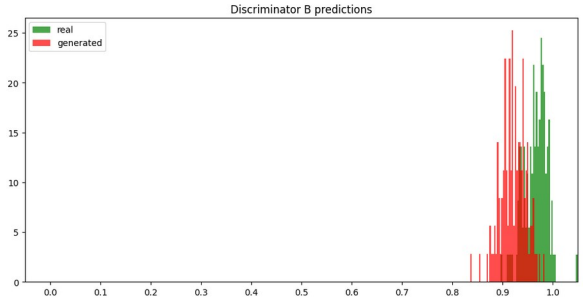
```
    draw_every = 1,
    chkp_folder = "./chkp",
    model_name = "cycle_gan",
    images_per_validation=3,
    plots=None,
    starting_epoch=0,
)
```

#50/50:









Images from A

Original from A

Translated to B

Reconstructed A

Original from A

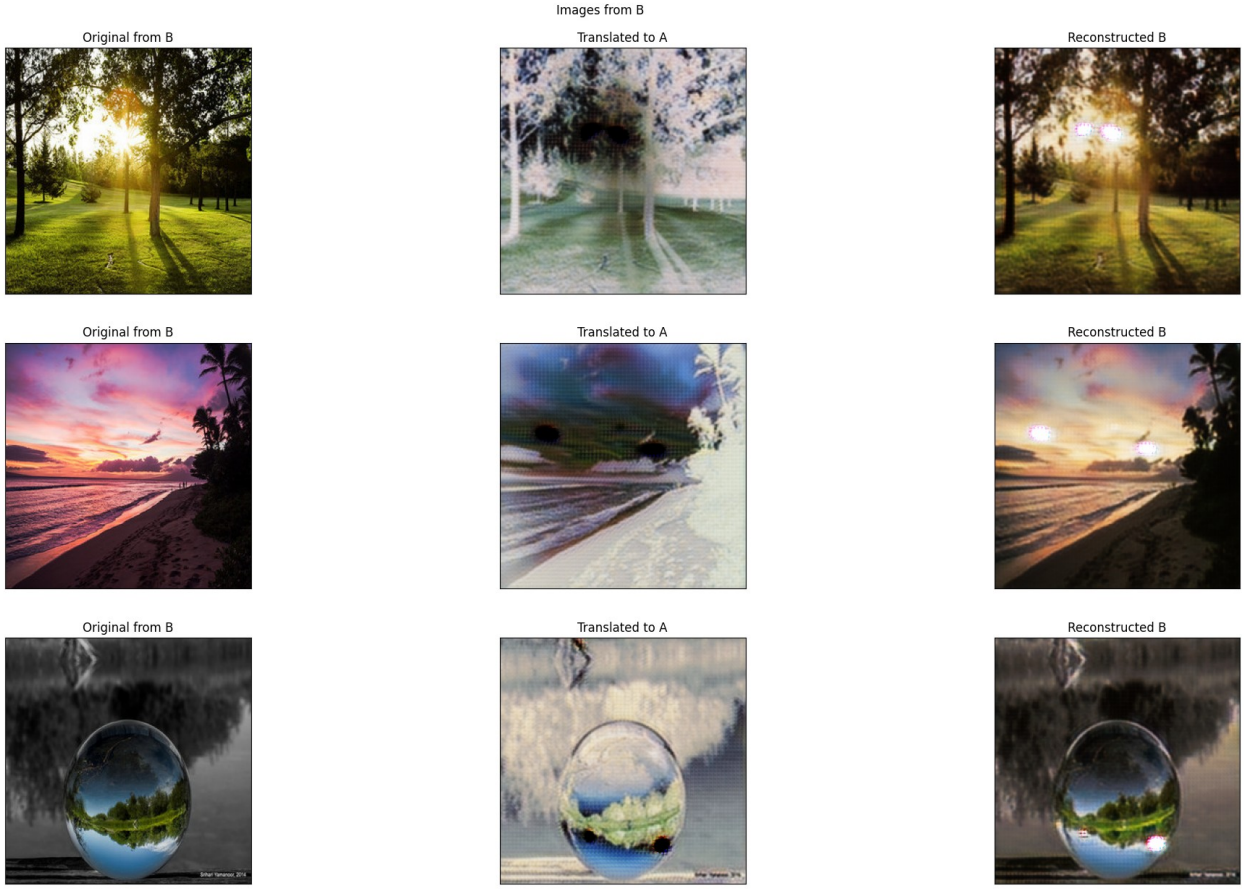
Translated to B

Reconstructed A

Original from A

Translated to B

Reconstructed A




```
CPU times: user 21min 40s, sys: 12min 28s, total: 34min 9s
Wall time: 5h 48min 11s
```

model

```
%%time
model, optimizer_d, optimizer_g, plots = learning_loop(
    model = model,
    optimizer_g = optimizer_g,
    g_iters_per_epoch = 1,
    optimizer_d = optimizer_d,
    d_iters_per_epoch = 1,
    train_loader_a = dataloaders.train_a,
    train_loader_b = dataloaders.train_b,
    val_loader_a = dataloaders.test_a,
    val_loader_b = dataloaders.test_b,
    criterion_d = criterion_d,
    criterion_g = criterion_g,
    scheduler_g = scheduler_g,
    scheduler_d = scheduler_d,
    de_norm_a = de_normalize_a,
    de_norm_b = de_normalize_b,
    epochs = 48,
```

```
    min_lr = 1e-6,
    val_every = 5,
    draw_every = 5,
    chkp_folder = "./chkp",
    model_name = "cycle_gan",
    images_per_validation=2,
    plots=None,
)
```

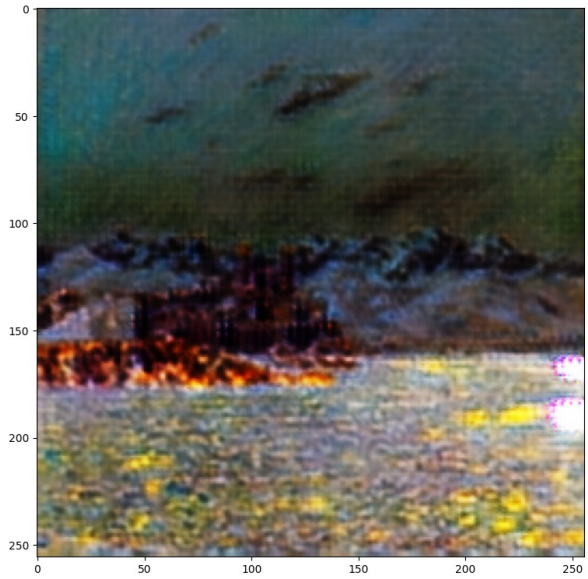
```
/var/folders/_f/ty_nth3s4zg__cf1rzb7fnz80000gn/T/
ipykernel_48264/3009969194.py:21: UserWarning: Selected model_name was
used already! To avoid possible overwrite - model_name changed to
cycle_gan#2
  warnings.warn(f"Selected model_name was used already! To avoid
possible overwrite - model_name changed to {model_name}")

#1/48:
train discriminators (1 times)

100%|██████████| 268/268 [01:09<00:00,  3.83it/s]

train generators (1 times)

100%|██████████| 268/268 [05:29<00:00,  1.23s/it]

#2/48:
train discriminators (1 times)

100%|██████████| 268/268 [01:09<00:00,  3.84it/s]

train generators (1 times)

 62%|██████    | 165/268 [03:24<02:07,  1.24s/it]

CPU times: user 43.8 s, sys: 24 s, total: 1min 7s
Wall time: 11min 13s
```

```
---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
Cell In[292], line 1
----> 1 get_ipython().run_cell_magic('time', '', 'model, optimizer_d,
optimizer_g, plots = learning_loop(\n    model = model,\n
optimizer_g = optimizer_g,\n    g_iters_per_epoch = 1,\n
optimizer_d = optimizer_d,\n    d_iters_per_epoch = 1,\n
train_loader_a = dataloaders.train_a,\n    train_loader_b =
dataloaders.train_b,\n    val_loader_a = dataloaders.test_a,\n
val_loader_b = dataloaders.test_b,\n    criterion_d = criterion_d,\n
criterion_g = criterion_g,\n    scheduler_g = scheduler_g,\n
```

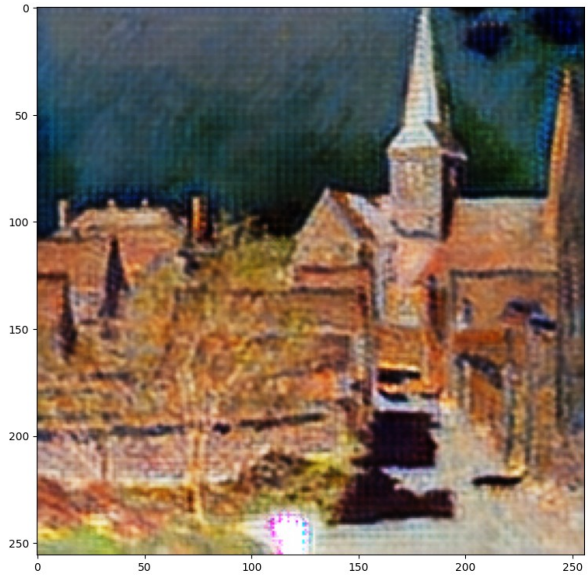
```
scheduler_d = scheduler_d,\n    de_norm_a = de_normalize_a,\n
de_norm_b = de_normalize_b,\n    epochs = 48,\n    min_lr = 1e-6,\n
val_every = 5,\n    draw_every = 5,\n    chkp_folder = "./chkp",\n
model_name = "cycle_gan",\n    images_per_validation=2,\n
plots=None,\n)\n')

File <timed exec>:1
----> 1 'Could not get source, probably due dynamically evaluated
source code.'

Cell In[282], line 90, in learning_loop(model, optimizer_g,
g_iters_per_epoch, optimizer_d, d_iters_per_epoch, train_loader_a,
train_loader_b, val_loader_a, val_loader_b, criterion_d, criterion_g,
de_norm_a, de_norm_b, scheduler_d, scheduler_g, min_lr, epochs,
val_every, draw_every, model_name, chkp_folder, images_per_validation,
plots, starting_epoch)
     86         # train generators
     87         print(f"train generators ({g_iters_per_epoch} times)")
     88         loss_g = []
     89         for _ in range(g_iters_per_epoch):
---> 90             model, optimizer_g, loss = train_generators(model,
optimizer_g, train_loader_a, train_loader_b, criterion_g)
     91             loss_g.append(loss)
     92         plots['train G'].extend(loss_g)
     93

Cell In[279], line 39, in train_generators(model, opt_g, loader_a,
loader_b, criterion_g)
     35             idt_a=idt_a,
     36             idt_b=idt_b,
     37         )
     38
---> 39         loss.backward()
     40         opt_g.step()
     41         losses_tr.append(loss.item())
     42

File
~/Makar/SHAD/venv/lib/python3.12/site-packages/torch/_tensor.py:631,
in Tensor.backward(self, gradient, retain_graph, create_graph, inputs)
    621 if has_torch_function_unary(self):
    622     return handle_torch_function(
    623         Tensor.backward,
    624         (self,),
   (...)    629         inputs=inputs,
    630     )
--> 631 torch.autograd.backward(
    632     self, gradient, retain_graph, create_graph, inputs=inputs
    633 )
```

```
File
~/Makar/SHAD/venv/lib/python3.12/site-packages/torch/autograd/__init__
.py:381, in backward(tensors, grad_tensors, retain_graph,
create_graph, grad_variables, inputs)
    376     retain_graph = create_graph
    378 # The reason we repeat the same comment below is that
    379 # some Python versions print out the first line of a multi-
line function
    380 # calls in the traceback and some print out the last line
--> 381 _engine_run_backward(
    382     tensors,
    383     grad_tensors_,
    384     retain_graph,
    385     create_graph,
    386     inputs_tuple,
    387     allow_unreachable=True,
    388     accumulate_grad=True,
    389 )

File
~/Makar/SHAD/venv/lib/python3.12/site-packages/torch/autograd/graph.py
:869, in _engine_run_backward(t_outputs, *args, **kwargs)
    867     unregister_hooks =
_register_logging_hooks_on_whole_graph(t_outputs)
    868 try:
--> 869     return Variable._execution_engine.run_backward(  # Calls
into the C++ engine to run the backward pass
    870         t_outputs, *args, **kwargs
    871     )  # Calls into the C++ engine to run the backward pass
    872 finally:
    873     if attach_logging_hooks:

KeyboardInterrupt:
```

```
img_a = ds.test_a[1].to(device).unsqueeze(0)

plt.subplots(1, 2, figsize=(20, 10))

plt.subplot(121)
plt.imshow(de_normalize_a(img_a[0]))

plt.subplot(122)
plt.imshow(de_normalize_b(model.generators["a_to_b"](img_a)[0]))

plt.show()
```



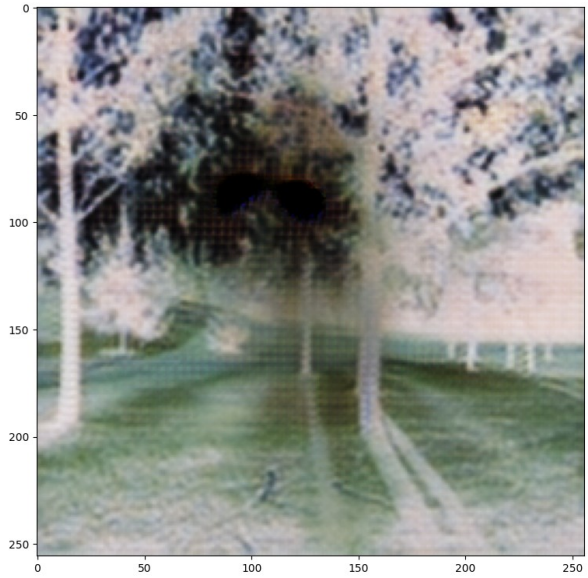

```
img_a = ds.test_a[5].to(device).unsqueeze(0)

plt.subplots(1, 2, figsize=(20, 10))

plt.subplot(121)
plt.imshow(de_normalize_a(img_a[0]))

plt.subplot(122)
plt.imshow(de_normalize_b(model.generators["a_to_b"](img_a)[0]))

plt.show()
```



```
img_ind = 1
img_a = ds.test_a[img_ind].to(device).unsqueeze(0)
fake_b = model.generators["a_to_b"](img_a)[0]
plt.subplots(1, 2, figsize=(20, 10))

plt.subplot(121)
plt.imshow(de_normalize_a(img_a[0]))

plt.subplot(122)
plt.imshow(de_normalize_b(fake_b))

plt.show()
```


```
img_ind = 0
img_b = ds.test_b[img_ind].to(device).unsqueeze(0)
fake_a = model.generators["b_to_a"](img_b)[0]
plt.subplots(1, 2, figsize=(20, 10))

plt.subplot(121)
plt.imshow(de_normalize_b(img_b[0]))

plt.subplot(122)
plt.imshow(de_normalize_a(fake_a))

plt.show()
```


0
50
100
150
200
250
0
50
100
150
200
250


0
50
100
150
200
250
0
50
100
150
200
250